

Atividade FrontEnd: TaskFlow - Gestão de Prioridades

Objetivo

Desenvolver uma aplicação React que gerencie tarefas permitindo a atribuição de níveis de prioridade, filtragem por status e persistência de dados.

Requisitos Técnicos

- **Gerenciamento de Estado:** Uso de objetos complexos no useState.
- **Hooks de Efeito:** Sincronização com LocalStorage.
- **Lógica de Filtro:** Implementação de visualizações parciais da lista original.
- **Estilização Condicional:** Alteração de cores e estilos baseada em propriedades do objeto (Prioridade).

1. Implementação do Código (App.jsx)

JavaScript

```
import React, { useState, useEffect } from 'react';
```

```
import './App.css';
```

```
function App() {
```

```
  const [taskText, setTaskText] = useState("");
```

```
  const [priority, setPriority] = useState("Baixa");
```

```
  const [taskList, setTaskList] = useState([]);
```

```
  const [filter, setFilter] = useState("Todas");
```

```
  useEffect(() => {
```

```
    const saved = localStorage.getItem("@taskflow_data");
```

```
    if (saved) setTaskList(JSON.parse(saved));
```

```
  }, []);
```

```
  useEffect(() => {
```

```
    localStorage.setItem("@taskflow_data", JSON.stringify(taskList));
```

```
}, [taskList]);
```

```
const addTask = (e) => {  
  e.preventDefault();  
  if (!taskText.trim()) return;
```

```
  const newTask = {  
    id: crypto.randomUUID(),  
    text: taskText,  
    priority: priority,  
    completed: false,  
    createdAt: new Date().toLocaleDateString()  
  };  
  
  setTaskList([newTask, ...taskList]);  
  setTaskText("");  
};
```

```
const toggleTask = (id) => {  
  setTaskList(taskList.map(t =>  
    t.id === id ? { ...t, completed: !t.completed } : t  
  ));  
};
```

```
const deleteTask = (id) => {  
  setTaskList(taskList.filter(t => t.id !== id));  
};
```

```
const filteredTasks = taskList.filter(t => {  
  if (filter === "Pendentes") return !t.completed;
```

```
if (filter === "Concluídas") return t.completed;

return true;

});

return (

  <div className="app-container">

    <header>

      <h1>TaskFlow</h1>

      <p>Gestão de Produtividade</p>

    </header>

    <section className="form-section">

      <form onSubmit={addTask}>

        <input

          value={taskText}

          onChange={(e) => setTaskText(e.target.value)}

          placeholder="Descrição da tarefa..."

        />

        <select value={priority} onChange={(e) => setPriority(e.target.value)}>

          <option value="Baixa">Baixa</option>

          <option value="Média">Média</option>

          <option value="Alta">Alta</option>

        </select>

        <button type="submit">Criar</button>

      </form>

    </section>

    <section className="filter-section">

      {[ "Todas", "Pendentes", "Concluídas" ].map(f => (

        <button
```

```
      key={f}

      className={filter === f ? "active" : ""}

      onClick={() => setFilter(f)}

    >

      {f}

    </button>

  )}

</section>

<main className="task-grid">

  {filteredTasks.map(item => (

    <div key={item.id} className={`task-card ${item.priority.toLowerCase()} ${item.completed
? 'done' : ''}`}>

      <div className="task-content">

        <h3>{item.text}</h3>

        <span>Prioridade: {item.priority}</span>

        <small>Criada em: {item.createdAt}</small>

      </div>

      <div className="task-actions">

        <button onClick={() => toggleTask(item.id)}>

          {item.completed ? "Reabrir" : "Concluir"}

        </button>

        <button onClick={() => deleteTask(item.id)} className="delete">

          Remover

        </button>

      </div>

    </div>

  )})

</main>

</div>
```

```
);  
}
```

```
export default App;
```

2. Estilização Estruturada (App.css)

CSS

```
:root {
```

```
  --low: #28a745;
```

```
  --medium: #ffc107;
```

```
  --high: #dc3545;
```

```
}
```

```
.app-container {
```

```
  max-width: 800px;
```

```
  margin: 2rem auto;
```

```
  padding: 0 1rem;
```

```
  font-family: system-ui, sans-serif;
```

```
}
```

```
header { text-align: center; margin-bottom: 2rem; }
```

```
.form-section form {
```

```
  display: flex;
```

```
  gap: 10px;
```

```
  background: #f8f9fa;
```

```
  padding: 20px;
```

```
  border-radius: 8px;
```

```
}
```

```
.form-section input { flex: 3; padding: 10px; }
```

```
.form-section select { flex: 1; }
```

```
.filter-section {  
  display: flex;  
  justify-content: center;  
  gap: 10px;  
  margin: 20px 0;  
}
```

```
.filter-section button.active {  
  background: #333;  
  color: white;  
}
```

```
.task-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));  
  gap: 15px;  
}
```

```
.task-card {  
  padding: 15px;  
  border-radius: 8px;  
  border-left: 8px solid #ccc;  
  background: white;  
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);  
  display: flex;  
  flex-direction: column;  
  justify-content: space-between;  
}
```

```
.task-card.baixa { border-color: var(--low); }

.task-card.média { border-color: var(--medium); }

.task-card.alta { border-color: var(--high); }

.task-card.done { opacity: 0.6; background: #e9ecef; }


.task-content h3 { margin: 0 0 10px 0; font-size: 1.1rem; }

.task-content span { display: block; font-size: 0.8rem; font-weight: bold; }


.task-actions {

  margin-top: 15px;

  display: flex;

  justify-content: space-between;

}


.task-actions button.delete { background: #ff4d4d; }
```

O que você como aluno expert e técnico em dev deve realizar?

Para tornar a atividade ainda mais completa, realize as seguintes implementações:

1. **Ordenação Automática:** Implementar uma lógica que mantenha as tarefas de prioridade "Alta" no topo da lista.
2. **Busca em Tempo Real:** Adicionar um campo de entrada que filtre as tarefas por texto conforme o usuário digita.
3. **Edição de Tarefas:** Adicionar um botão "Editar" que transforme o texto da tarefa em um input para alteração da descrição original.
4. **Confirmação de Exclusão:** Implementar um window.confirm ou um modal customizado antes de deletar uma tarefa definitivamente.

Esta estrutura foca em **Componentização e Lógica de Negócio**, preparando para cenários reais de desenvolvimento Front-end.